ID2216 Group 10 Project Submission

Group Members

S/N	Name	Canvas ID
1	RAMEESHA HUSSAIN	175822
2	YEN YI CHEN	211895
3	SIOW TERNG JUEN	211444
4	LO WEI SHEN	211541

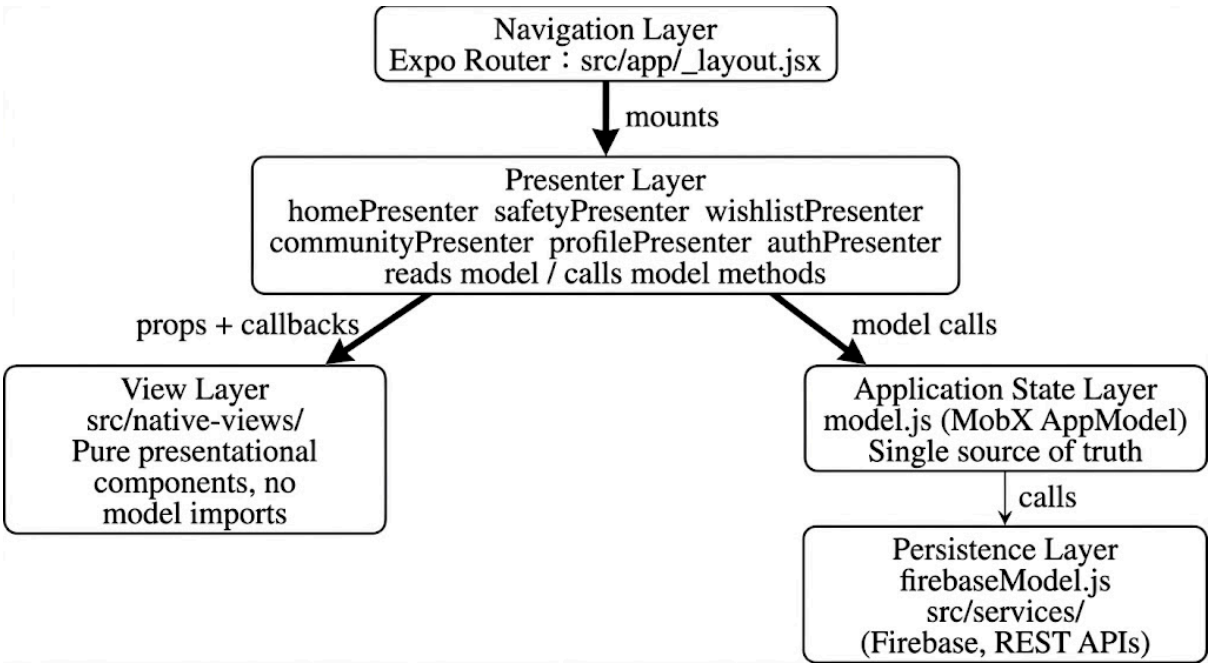
1. Application Overview

- App name: Solo Buddy
- Platform: React Native + Expo (iOS/Android native app)
- Purpose: SoloBuddy is a travel companion app built for solo travellers. It combines destination exploration (nearby attractions, maps, search), suggested itinerary creation, a community feed and a dedicated Safety tab surfacing official travel advisories, local news and real time local weather. Save places to your trip wishlist, track what you have visited, and head somewhere new with confidence.
- Target group: Travellers who want consolidated safety and discovery info in one app
- Deployed Link: https://expo.dev/accounts/tsiow/projects/solobuddy/builds/ae6893a0-2254-45e9-999b-ab75d69bb4a7
- Github Link: https://gits-15.sys.kth.se/iprog-students/rameesha-siow-wslo-yyche-vt26-project

2. Architecture

Layer	Our Implementation
Persistence	firebaseModel.js: Firebase Firestore, Auth, Storage
Application State	model.js: MobX AppModel class (makeAutoObservable)
Presenters	src/presenters/ (one file per feature)
Views	src/native-views/ (one file per view)
Navigation	Expo Router (file-based, src/app/)

Architecture Diagram



3. Web APIs and Persistence

External APIs

A. Google Places API: placesService.js

The app uses three endpoints from the Google Places API v1:

- Nearby Search (places:searchNearby): Fetches up to 20 tourist attractions, museums, and parks within a 5 km radius of the user's GPS coordinates.
- Text Search (places:searchText): Powers the map page's free-text search, biased toward the user's current location.
- Place Details (places/{placeId}): Retrieves rich details for a selected attraction, including opening hours, phone number, website, and user rating count.

Each result's cover photo is fetched separately via the Places Photo API. All raw responses are normalized into a consistent internal shape by normalizePlaceACB(), decoupling the view layer from the API's response format.

B. Google Weather API: weatherService.js

Three Google Weather-related endpoints are called, using Promise.all to avoid sequential blocking:

- Current Conditions (weather.googleapis.com/v1/currentConditions): Real-time temperature, weather description, feels-like temperature, humidity, wind speed, and UV index.

- Daily Forecast (weather.googleapis.com/v1/forecast/days): 7-day forecast including daily high/low temperatures, precipitation probability, and sunrise/sunset times.
- Air Quality (airquality.googleapis.com/v1/currentConditions): Current Air Quality Index (AQI) with a human-readable category label (e.g., "Good", "Unhealthy").

The service exposes two aggregated functions, fetchWeatherBannerACB for a compact summary card and fetchWeatherDetailsACB for the full detail view, keeping the presenters simple and the API logic fully encapsulated in the service layer.

C. UK Foreign Travel Advisory: ukAdvisoryService.js

The app queries the UK Government's public content API (gov.uk/api/content/foreign-travel-advice/{countrySlug}) to retrieve:

- Official travel alert status for the destination country
- Last updated timestamp
- Link to the full official advisory page

This API requires no authentication key and is fully open. The data is displayed in the Safety screen alongside the US advisory to give users a dual-source safety picture.

D. US State Department Travel Advisory: usAdvisoryService.js

The US Department of State Travel Advisory API (cadataapi.state.gov) provides a standardized safety level (Level 1–4) and a short label such as "Exercise Normal Precautions" or "Do Not Travel". Due to CORS restrictions on the client side, requests are proxied through the course-provided KTH proxy server (brfenergi.se/iprog/group/541/).

E. NewsAPI: newsService.js

The app queries newsapi.org/v2/everything using the destination name as a keyword to surface the 10 most recent English-language news articles sorted by publication time. This gives travelers a real-time awareness of local events or incidents at their intended destination that may not yet appear in official advisories.

F. OpenRouter AI API: openRouterService.js

When a user has at least 5 unvisited attractions in their wishlist, the app calls the OpenRouter API (openrouter.ai/api/v1/chat/completions) using the openai/gpt-oss-120b model to generate a personalized multi-day travel itinerary. The prompt instructs the model to cluster attractions by geographic proximity, minimize daily travel time, and return structured JSON containing a day-by-day plan with suggested visit times and practical tips per stop. Image URLs and verbose fields are stripped from the input to reduce token usage. The model's JSON response is parsed and rendered directly on the Wishlist screen.

Firebase Persistence

All persistence is handled through three Firebase services, initialized in firebaseModel.js.

Firebase Authentication

Email and password authentication is implemented using Firebase Auth. An onAuthStateChanged listener keeps the MobX model's currentUser field in sync with the active session at all times, including across app restarts. On sign-out, all user state is cleared from the model.

Firebase Firestore

The Firestore database uses the following structure:

- /users/{uid}: stores each user's profile (name, email, birthday, phone number, avatar URL)
- /users/{uid}/wishlist/{attractionId}: stores the user's saved attractions as a subcollection, including coordinates, visit status, and creation timestamp
- /communityPosts/{postId}: a shared top-level collection for community posts, including author info, location tag, category, and a likedBy array

Data isolation is enforced by design: the wishlist is stored as a subcollection under each user's own document, so no user can access another user's saved places.

Both the wishlist and community posts collections use Firestore's onSnapshot for real-time listening. This means that if the same user opens the app on two devices simultaneously, any change (adding an attraction, posting a tip, liking a post) is reflected instantly on both devices without requiring a manual refresh, satisfying the A-plus criterion for live persistence updates.

The toggleLikePost function in communityService.js uses Firestore's arrayUnion and arrayRemove atomic operations to update the likedBy field. This prevents race conditions when multiple users like the same post at the same time, satisfying the A-plus criterion for collaborative features with conflict management.

Firebase Storage

Profile photos are uploaded to Firebase Storage under users/{uid}/profile-photo-{timestamp}.jpg. After upload, the download URL is written back to the user's Firestore profile document, keeping the photo reference consistent across sessions and devices.

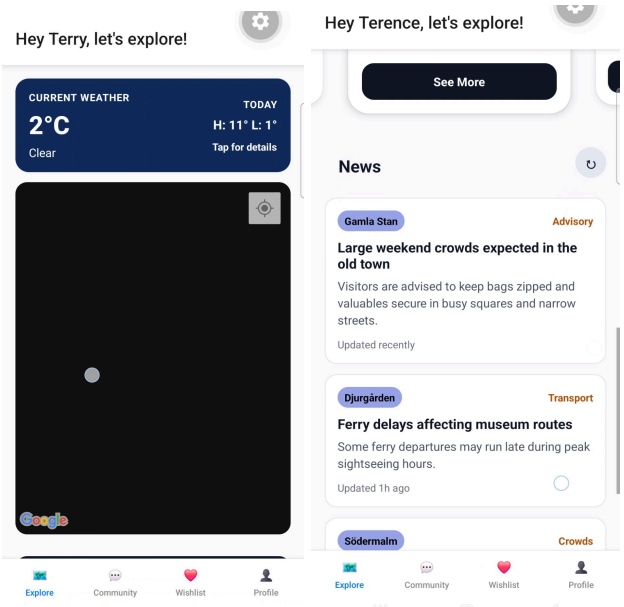
4. Application Showcase

Feedback from mid-project review:
Anonymous student 1 - “I like the idea of safety help from different kinds of statistics, (however) the idea of showing local news seems a little excessive unless you mean things like, active natural disasters or for example the gang war problems in Mexico. Under normal conditions it feels like

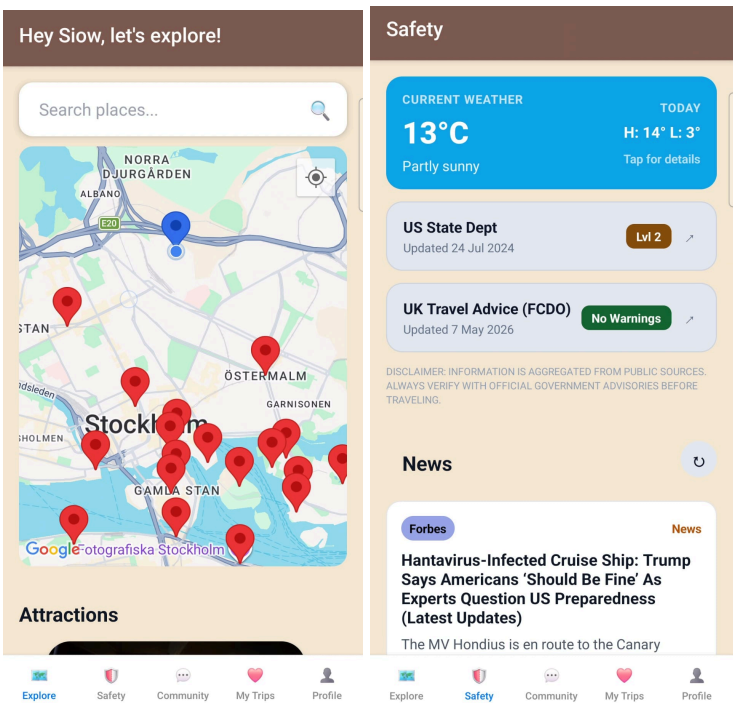
maybe it could be a little cluttered if too much information is given.”

Anonymous student 2 - “If you have time, make the UI look more colorful. It lights people’s hearts when traveling.”

Screenshots of our app before the feedback:



Before, the homeView had everything, from weather updates to news and weather alerts. As the user-stated, it felt too cluttered and some info was too excessive to show up every time. As a result, we changed the layout of our app, moving most of the “safety” related data into a new “safety” tab. We also improved the design of our app to make it more colourful and lively as shown below:



5. Usability and User Experience

The application was designed with a strong focus on simplicity, accessibility, and reducing information overload for travelers. Since users may interact with the app while moving, navigating unfamiliar locations, or under time pressure, the interface prioritizes quick access to important information through clear navigation and lightweight interactions.

The app uses a bottom-tab navigation structure implemented with Expo Router, separating the application into clear functional areas such as Home, Map, Safety, Wishlist, Community, and Profile. This reduces cognitive load by ensuring each screen has a single primary purpose.

Several usability improvements were made after the mid-project feedback session. Initially, the home screen displayed weather updates, advisories, and news simultaneously, which made the interface feel crowded. Based on peer feedback, safety-related information was moved into a dedicated Safety tab, while the Home screen was simplified to focus on attraction discovery and quick summaries. This improved readability and reduced unnecessary information exposure for casual browsing.

The user interface was also redesigned to feel more visually engaging and travel-oriented. The app icons were also drawn using a separate app to fit the design and the color palette used in the application design. We introduced:

- more colorful cards and icons,

- consistent spacing and typography,
- larger touch-friendly buttons,
- weather-based visual banners,
- and simplified card layouts for attractions and community posts.

The app also supports personalization features that improve user experience:

- users can upload a profile picture,
- save attractions to a wishlist,
- mark attractions as visited,
- generate AI-based itineraries,
- and interact with community travel posts.

Special attention was given to mobile usability. Components were designed for React Native touch interaction using Pressable elements, scrollable card layouts, and responsive sizing that works across different phone screen sizes. The map search and attraction cards were optimized to minimize excessive typing and navigation steps.

Overall, the project evolved significantly through iterative feedback and usability-focused redesigns, resulting in a cleaner, more organized, and more engaging travel companion experience.

6. Group Cooperation

Name	**Primary role**
RAMEESHA HUSSAIN	Visual design, icons, layout, and color iteration
YEN YI CHEN	Community feature, map search, service layer refactoring
SIOW TERNG JUEN	Weather feature, map UI, profile system, UI polish
LO WEI SHEN	Core infrastructure, external APIs, Firebase Auth, and AI feature